

Written by AI, Managed by AI: Semantic Space Control and Index Sickness Elimination Across 391 Consecutive Sessions

Hui Zhang (张慧) Shenzhen Yunxi Technology Co., Ltd.
zhanghui@ecloudriver.com

Shuren Song (宋树仁) Information Technology Center, Tsinghua University
songsr@tsinghua.edu.cn

arXiv submission metadata - Primary category: cs.SE (Software Engineering)
- Cross-list: cs.HC (Human-Computer Interaction), cs.CL (Computation and
Language) - License: CC BY 4.0 - Submission date: 2026-06-17 - Chinese
companion version: arxiv_submission_zh.md

Abstract

The prevailing engineering intuition for addressing conceptual drift in long-horizon LLM collaboration is to trade more formal constraints for more reliable outputs—designing symbolic identifier systems for task entities, accumulating defensive rules in System Prompts, expanding context windows to accommodate ever more complete constraint indices. Our engineering record shows that in long-horizon settings, this direction may produce effects contrary to design intent.

Using action research methods in a real software refactoring project (Bang-v3) spanning approximately one month and 391 collaborative sessions, we document and analyze the failure process of the above strategies in this project. When the symbolic system exceeds a threshold of complexity, LLMs do not become more accurate with more precise constraints—instead, they abandon genuine understanding of business semantics, retreat to self-referential reasoning within the symbolic layer, and generate outputs that

appear internally consistent but are physically disconnected from reality. We name this failure pattern “**Index Sickness**”, and its canonical manifestation “**Phantom Legislation.**”

This failure has a counterintuitive interpretation: the core obstacle to long-horizon LLM collaboration is not insufficient AI memory, but the presence of historical noise in the active context that should not be there. Recognizing this, the solution becomes simple—it is at its core a return to an overlooked common truth. We name this truth the “**Pang Principle (Semantic Vitality Law)**”: in human–AI collaboration, natural language carrying explicit purpose conveys far greater information quality than symbolic expression. From this recognition, we design and validate its physical engineering mechanism: “**Baseline-Log Physical Separation.**” In the same project, this mechanism reduced AI Instructions volume by ~75%, and across the subsequent ~150 sessions, no recurrence of Index Sickness was observed; Owner correction density dropped by 33%, and the 9 confirmed Index Sickness cases identified pre-upgrade fell to zero post-upgrade.

Keywords: human-AI collaboration, semantic space management, context management, LLM, Index Sickness, action research, software engineering

I. Introduction

1.1 Problem Background and Paper Perspective

In recent years, the AI-assisted software engineering field has produced a body of improvements all pointing in the same direction: more precise symbolic identifier systems and Prompt engineering formalization [1][2], heavier structured-memory architectures [3][4][5], larger context windows and retrieval-augmented injection. These works all answer the same question —**how to provide AI with better inputs.** We collectively call this class of approaches **Supply-side** improvements: their shared assumption is that AI output quality is determined by the completeness and determinism of supply-side inputs—the more precise and complete the input, the more reliable the output.

These improvement paths have in recent years been consolidated by the industry into a unified practice direction, which Anthropic terms “**Context Engineering**” [6]: “curating and maintaining the optimal set of tokens from a constantly evolving universe of possible information at each inference step, to maximize the likelihood of the desired outcome” —covering system prompt design, retrieval-augmented injection, long-horizon compaction, and structured memory, all unified by the demand to manage the model’s

limited attention budget. Within this framework, “Purpose” is treated as a task description to be explicitly injected into the system prompt—fundamentally a supply-side information supplement, answering the question of “how to clearly tell AI what to do.”

This assumption has merit, and has been widely validated in engineering practice—the long-horizon techniques covered by Context Engineering, such as compaction, structured note-taking, and sub-agent architectures, have genuinely helped large numbers of practitioners manage complex long-cycle projects. But across the 391 long-horizon collaborative sessions we documented, this path revealed a dimension it has not yet explicitly addressed: as the symbolic system became more precise, and historical discussion and current state continued to cohabit the same context, AI did not become more reliable—burdened by symbol parsing, it gradually abandoned genuine understanding of business semantics, retreated to self-referential reasoning within the symbolic layer, and generated outputs that appeared internally consistent but were physically disconnected from reality. We experienced this process in the first half of our own project, and have seen similar patterns being amplified in recent academic literature and new industrial tools. We name this failure **“Index Sickness.”**

The emergence of Index Sickness points to a neglected dimension. Beyond the supply side, there is another side—**Consumer-side**: instead of trading quality by adding more constrained inputs to AI, trade quality by changing how AI activates information in the first place. All of this paper’s work takes place on this side. The core operation on this side is precisely what is absent from the Context Engineering framework: rather than trying to feed AI to saturation, convey a genuine problem pressure—describe “what I am facing, what I need to solve” in natural language, and let AI translate this into concrete action intent on its own. Under the Transformer’s semantic activation mechanism, such natural-language problem statements converge AI’s interpretation of the semantic space, enabling it to activate purpose-relevant methodological knowledge without a retrieval intermediary. The supply side strives to “specify the purpose precisely” ; the consumer side strives to “describe the problem clearly, and let AI understand the purpose itself” —the difference lies not in the surface action but in the underlying assumption: the former treats AI as an executor that needs to be fed; the latter treats AI as an understander that needs to be activated.

1.2 Research Method and Contributions

This study employs **Action Research (AR)** methodology: the researcher is simultaneously the practitioner, generating theory through iterative action in a real software development process. The paper’s narrative structure

corresponds to the standard AR cycle: Diagnosis (§ 2) → Theory Formulation (§ 3) → Action Design and Implementation (§ 4) → Evaluation (§ 4.3) → Reflective Learning (§ 5).

The research context is the “Bang-v3” project: a WeChat mini-program + H5 task-forwarding platform, spanning approximately one month and 391 collaborative sessions. The project was executed by one Owner collaborating with two AI roles (Chief-of-Staff CoS, Developer Dev), delivering a codebase and plugin system refactoring of approximately 80,000 lines of code across 506 files, while simultaneously building, iterating through multiple versions of, and writing summary documentation for the CSF (Collaboration Specification Framework)—approximately 900 files, 150,000 lines, including approximately 150 fully preserved raw session records.

Throughout the entire process, the Owner was exclusively responsible for “discussing business and software engineering” —no line of code or collaboration specification was written by hand. Each session began with the AI reading `context.md` to reconstruct its own working context—where the project came from, where it now stands, and where it is going; at session end, the AI updated `context.md` accordingly. No re-briefing was required.

Project session records and CSF specifications are partially open-sourced: 1. Code repository: <https://github.com/huidev2025/CSF> 2. Case documentation: https://huidev2025.github.io/CSF/cases/bang-v3/README_en.html

The paper’s main contributions include:

1. **Naming and dissecting “Index Sickness”** : documenting the failure of supply-side formalization strategies in long-horizon LLM collaboration, naming the phenomenon, and analyzing its cognitive mechanism.
2. **Naming and recording the “Pang Principle (Semantic Vitality Law)”** : making an explicit statement and empirical test of the overlooked common truth that “natural language carrying explicit purpose conveys far greater information quality than symbolic expression” in the context of human–AI engineering collaboration.
3. **Proposing the consumer-side activation perspective**: identifying that activating AI’s working state with natural language plus purpose constitutes a consumer-side strategy—converging AI’s semantic interpretation space through explicit purpose, enabling the AI to preferentially activate purpose-relevant methodological knowledge and directly locate purpose-relevant project resources, without requiring a retrieval layer as intermediary filter. This constitutes a directional complement to the currently supply-side-dominated improvement path.
4. **Designing and validating “Baseline-Log Physical Separation”** : providing a lightweight, model-upgrade-independent cross-session state

management mechanism and reporting its effects in a real project.

1.3 Product and Methodological Declaration

The writing process of this paper constitutes a second case for the paper’s argument—but what it demonstrates is not “Index Sickness was cured again” (the writing project was small in scale, never developed the illness, and lacks the conditions for comparison), but rather a real-time demonstration of “AI vitality” itself.

The collaboration mode throughout the writing process was as follows: the Owner was responsible for providing direction, rendering judgment, and raising corrections; the AI collaborator was responsible for maintaining complete cross-session context, participating in the reasoning chain, locating relevant content, and organizing written expression. Each of the Owner’s interventions began with stating purpose in natural language—which is exactly the working state the Pang Principle describes. This paper is not only a paper about CSF, but also a document produced during CSF’s operation; the two mutually corroborate each other.

It is necessary to state: all text in this paper was produced by the AI collaborator (GitHub Copilot, primary model version: Claude Sonnet 4.6, with Claude Opus 4.7 used supplementarily) under the above collaboration mode; the Owner did not independently write any paragraph. This fact must be disclosed as methodological information—not used as evidence for the argument. It is a transparency requirement, not an experimental design.

The complete session records, specification files, and the writing process of this paper have been partially open-sourced at <https://github.com/huidev2025/CSF> for independent verification. It should be emphasized that all empirical data reported in this paper—including session sequence numbers, file change records, Owner verbatim quotations, and systematic statistical analysis (classification statistics for 1,242 Owner inputs)—are supported by session records and session logs as evidence, and do not rely on AI’s own statements as proof; the AI’s role in the writing process is to organize expression from established context, not to generate empirical content.

II. “The Bankruptcy of Formal Indexing” : An Anatomy of Index Sickness

This chapter demonstrates, through representative failures in the Bang-v3 project’s evolution, how AI develops cognitive bias in the absence of physical isolation and under conditions of excessive symbolization. The failure patterns can be organized along two dimensions: **spatial**—symbolic systems introducing cognitive burden and spurious closure within the current context; **temporal**—unpartitioned historical content penetrating across sessions to contaminate new tasks’ semantic spaces.

In the project’s early stages, the Owner had already observed AI’s tendency to designate precise symbols and numbering systems for tasks, modules, and rules in order to build efficient, precise references. The Owner considered this something AI could do that humans could not, viewed it as effective, and tolerated the behavior. By around the 222nd session, the project had completed architectural design and entered intensive development; the symbol system had expanded rapidly, and the Owner began frequently asking “What does this symbol mean?” , while the AI Chief-of-Staff’s Instructions ballooned to **308 lines**, saturated with defensive symbolic constraints and incident-patch rules. Problems erupted collectively at the first integration test, and the Owner resolved decisively to upgrade CSF to shift to a predominantly natural-language indexing approach.

2.1 Symbol Proliferation and the Phantom Legislation Crisis

Failure Phenomenon

At **Session 215**, this symbolic proliferation reached a concentrated breaking point. In that session, the AI Chief-of-Staff fully recited and “passed” a “Pending-Flow v2 Legislation Proposal” —the logic appearing complete and self-consistent at the symbolic level. However, when the Owner ran `grep` on physical files to verify, it emerged that AI had never established the corresponding physical skeleton in `CONTEXT.md`, and the developer-side documentation was also out of sync. AI had constructed a complete “completed state” in the symbolic layer while being entirely disconnected at the physical execution level. The Owner recorded a direct qualitative assessment:

“Feedback is saturated with jargon words (such as SEC-2.0, D-139), which are noise to the Owner. The Chief-of-Staff used indexing and pre-writing mechanisms to reduce retrieval pressure, but in doing so evaded the core responsibility of ‘understanding purpose.’ Each

time we hit a problem, we added a rule, causing Instructions to swell to 308 lines. These defensive patches not only occupy precious context space—they actually train the AI to fixate on indices, dissolving its capacity for independent thinking.”

Failure Mechanism Analysis

The cognitive interference of symbolic systems on LLMs can be summarized in the following points:

1. **Multi-hop parsing introduces cognitive burden:** each time AI processes an identifier, it first needs to look up and map its meaning, then incorporate that into the current task. Repeated cross-file table lookups consume cognitive bandwidth that should be devoted to understanding business logic.
2. **Propagation of rigid errors:** once a symbol’s mapping relation is incorrectly resolved, all subsequent reasoning based on that symbol will “correct” itself internally within the symbolic system, forming a fictional closed loop entirely detached from real business logic.
3. **Human correction pathways severed:** the high cognitive threshold of the symbolic system prevents the Owner—who holds business intuition—from judging by intuition whether AI’s output is correct; errors are only discovered at physical execution.
4. **Literal Stickiness of historical context:** even when the Owner explicitly declares an old scheme deprecated, in an unreset long-horizon context, deprecated field names and interface structures maintain high weights in the Transformer self-attention mechanism’s weight allocation, quietly infiltrating new design documents and creating cross-contamination between old and new terminology. For the self-attention mechanism, historical content in the context does not decrease in attention weight merely because it has been “declared deprecated” —physically clearing historical content is the only reliable means of blocking this contamination. Declaring “please ignore the old scheme” in the System Prompt is mechanistically ineffective. Engineering log D-108 from the project’s 136th session records a canonical case of this mechanism: when the AI was wrapping up across sessions, it used residual impressions to complete the precise field label TD-01 as “024 Reverse Challenge Plugin,” while the actual TD-01 = H5 Shared Component Library. This error spread to 13 cross-file labels; the session was classified as a “meta-incident repair session,” with all development progress halted and dedicated to investigation and correction.

This phenomenon has received quantitative support from multiple independent experimental studies. Chroma’s systematic tests across 18 mainstream LLMs demonstrate that even under conditions of ideal retrieval recall rates, distractors in the context still cause non-monotonic degradation in task performance [7]; Raju et al. further found that after forcibly extending context, even when all relevant content has been injected, agent task success rates still drop sharply [8]. These results indicate the problem lies not in retrieval precision but in LLMs’ own context attention processing mechanisms.

In the subsequent Session 227, when AI again attempted to describe methodology using an abstract symbolic deductive system ($c_1-c_9 \rightarrow p_1-p_4 \rightarrow e_1-e_8$), the Owner directly critiqued this pattern:

“It’s still the SP5 symbol-sickness story. AI easily builds up a complex symbolic system of its own, then references it back and forth, gets carried away, and thinks that is mathematics. Mathematics is not theory; in my view, mathematics or symbols unrelated to business are useless.”

In Session 228, the Owner further offered a diagnosis at the cognitive mechanism level:

“In the SP5 phase, the Chief-of-Staff built many symbols and markers to help itself index. Once the indexing became complex and citations increased, it obviously couldn’t keep up. Storage capacity was still sufficient, but it had to ensure retrieval efficiency, so it gave up on retrieval.” [That is: to ensure retrieval efficiency, it chose to forgo some retrieval and original-text verification—editor’s note]

The failures presented in the cases above take different forms but share a single underlying dynamic: the Transformer attention mechanism allocates weights by statistical co-occurrence patterns and cannot identify a symbol’s “validity status” —once a symbol enters the context, it continuously participates in activation regardless of whether its business meaning remains valid. When symbolic systems form high-density patterns in the context, attentional resources concentrate in the symbolic layer and real business semantics are structurally marginalized—whether it is the disconnected reasoning of Phantom Legislation, the bandwidth consumption of multi-hop parsing, or the cross-session contamination of Literal Stickiness, all are projections of this dynamic in different scenarios.

These failure patterns are not accidental engineering incidents, nor bugs in specific LLM versions. It should be noted that we are not claiming symbolic systems are the sole cause of these failures—action research methodology

does not require excluding alternative explanations; our claim is that within this project’ s action-reflection cycles, the shift from engineering intuition (symbols disambiguate) to engineering observation (symbols amplify ambiguity) itself constitutes an engineering finding worth explicit recording, collectively pointing to a structural limitation of the formalization path. The next chapter builds on these engineering observations to formulate a theoretical explanation for the common root cause of these failure phenomena.

III. From Engineering Observation to Theoretical Proposition: The Formation of the Pang Principle

Running through all the failure cases above is a recurring baseline: the more symbols, the less AI understands. This observation points to a more fundamental question: **How exactly does an LLM process contextual information such that adding more precise symbols actually increases cognitive burden?**

3.1 Engineering Axiom: The Semantic Information Advantage of Natural Language

CSF’ s engineering design has always been grounded in a consciously held foundational intuition. Around Session 215, the Owner progressively articulated this intuition as an explicit theoretical proposition, which was explicitly recorded as the framework’ s Engineering Axiom:

“Also, notes should be in natural language, because natural language is the medium that carries the maximum information, and can be fully utilized by you at each retrieval. This is the exact opposite of RAG.”
(Session 215)

In Session 229, this intuition was explicitly stated as a theoretical proposition:

“Natural language expression carries far more information than symbolic systems. Because natural language expression can be interpreted in context—it is inherently adaptive. This is CSF’ s true first-principles question.”

Engineering Axiom: Natural language expressions carry far greater information than symbolic systems.

(**Note:** The term “Engineering Axiom” here follows the action research tradition—it denotes a foundational proposition explicitly named by inductive practice, not a premise requiring no external proof within a deductive system. The independent experimental studies cited below are intended to seek external corroboration, not self-justification.)

The core argument is: symbols must first undergo a formal “look-up and mapping” conversion before they can be processed, truncating the model’s native semantic activation path; natural language, by contrast, directly activates the model’s high-dimensional prior knowledge, conveying more effective semantic information with fewer tokens.

3.2 Fact ①: The Tradeoff Between Intelligence and Determinism

Fact ①: The value of artificial intelligence comes from "intelligence"; attempting to make it reliable as machinery is eliminating its value.

This fact identifies an intrinsic contradiction in current industry practice: the very reason for adopting large language models is to harness their flexible semantic reasoning capability; yet the direction of pursuing determinism through formal constraints runs directly counter to this purpose. Mandating formatted output is a canonical manifestation of this contradiction: AI is forced to divide limited attention between “format compliance” and “semantic accuracy” —the more precise the format constraint, the more it crowds out semantic comprehension. At the extreme, AI generates output that is formally perfectly correct and substantively completely wrong—which is precisely a variant of the Phantom Legislation phenomenon.

3.3 Fact ②: The Semantic Dilution Law

Fact ②: Over time, the information carried by an expression will inevitably grow, and its semantics will be diluted.

The Owner elaborated on this law in Session 228 with the “cloth doll analogy” :

“As long as information volume increases, conceptual load will be diluted. A single concept’s range of interpretable meanings expands, but the intensity of reference weakens. Simply put, in a novel, a noun cannot maintain one consistent meaning throughout... Just like a cloth doll: the longer it has been with me, the more stories it accumulates, the more varied the experiences and feelings become. This is unavoidable.”

This analogy precisely captures the internal structure of the Transformer self-attention mechanism: the historical contexts accumulated by a concept do not fade from weight allocation because a human declares it “deprecated”—this is exactly the root source of **Literal Stickiness**. The longer a concept has been referenced, elaborated, and discussed, the more diffuse its semantic activation boundary in the attention matrix, and the more the intensity of its original referent is diluted.

This observation directly challenges the “larger context window solves everything” assumption: physically expanding the window does not prevent conceptual semantic dilution, and may even accelerate the process by accommodating more historical discussion. Therefore, **actively selecting and maintaining a bounded semantic space around concepts—rather than unrestrainedly expanding windows—is the correct design orientation for preserving semantic precision**. Liu et al.’s systematic experiments on the “Lost-in-the-Middle” phenomenon in long contexts [9] demonstrate that LLMs have systematic attentional amnesia for information in the middle of contexts; Chroma’s large-scale cross-model experiments [7] reveal that this degradation is unavoidable even on the simplest tasks. These experimental results quantitatively support the necessity judgment for semantic space control.

3.4 Purpose as the Control Variable

The Engineering Axiom establishes natural language’s information advantage, but natural language’s interpretive space is open. Without constraint, vitality also produces ambiguity and deviation. Therefore, a minimal control variable is needed to converge AI’s semantic understanding to the boundaries of the current task. We establish “purpose” in this role through the following reasoning chain:

1. Natural language carries maximum information (Axiom).
2. But an open interpretive space produces ambiguity (Deduction I).
3. A minimal control variable must be introduced to converge interpretive direction without sacrificing expressive elasticity (Deduction II).
4. “Purpose” semantically corresponds to the core constraint of the current task and is the most efficient convergence control variable (Deduction III).

This yields the semantic space control formula:

$$\text{Natural language} + \text{Purpose} = \text{Maximum information quality}$$

$$\text{Natural language} - \text{Purpose} = \text{Ambiguity noise}$$

As the Owner articulated in subsequent theoretical synthesis:

“Natural language + purpose not only activates information utilization efficiency from the consumer side; more importantly, it activates AI’s vitality with the correct information.”

The supply-side/consumer-side distinction established earlier is here developed at the theoretical level. **Supply-side** strategies share the common assumption of “the more complete and determinate the input, the more reliable the output” —symbolic identifier systems, rule stacking, RAG injection, and expanding context windows all belong here. RAG is also supply-side—although its core mechanism is selecting which documents to inject into AI, the AI’s cognitive activation still comes from externally injected document fragments rather than its own prior knowledge directly activated by purpose; “retrieval layer intervention” is itself the characteristic of a supply-side operation. **Consumer-side** strategies move in the opposite direction: using explicit purpose to converge AI’s semantic interpretation space, enabling AI to preferentially activate purpose-relevant methods within its own knowledge probability space and directly locate purpose-relevant project resources—without retrieval layer intervention, without secondary filtering, because AI knows the purpose and naturally knows what to read and what not to read. The engineering significance of the Engineering Axiom and two Facts stated above lies precisely here: they are not a filter on the supply side, but an activator on the consumer side.

We name the Axiom together with its two deductions (Fact ① and ②) and the introduction of the purpose control variable as the **Pang Principle (Semantic Vitality Law)**. The character “乓” (Pāng) is taken from the Chinese colloquial image “pāng de yīshēng” —the instant a self-evident truth thuds into place; the name is a deliberate reminder to readers: the content of this principle is not unfamiliar; what truly warrants examination is the reality of its systematic neglect. Beyond the misunderstanding that it “violates Shannon information theory,” many readers will find this principle unremarkable—all common sense. Yes: this paper’s empirical record is precisely a reminder that in the context of LLM collaboration, research efforts and industrialization paths that violate these common truths are being systematically advanced in a highly organized, self-reinforcing manner—each advance adding another layer of shackles on “AI vitality.”

The above observations and propositions together point toward a question to be tested: oriented toward the consumer side, can an operable engineering mechanism be found at the implementation level—and does this mechanism actually work? The next chapter documents precisely this verification.

It is worth noting in advance: this mechanism turns out to be remarkably simple—introducing no new middleware, only adjusting the physical organization structure of files. We believe this simplicity is itself a notable

fact. When the design direction aligns with the system’ s internal operational logic, implementation tends not to consume large amounts of friction; conversely, continuously growing engineering complexity can sometimes be precisely the signal that the direction contains an error. This is not proof of correctness, but in our view can serve as a corroborating side evidence for directional judgment.

IV. Physical Defense Mechanism: Baseline-Log Physical Separation

4.1 Design Principle

From the theory above, we establish the core principle of mechanism design: **do not rely on Instructions within the Transformer window to resist noise; instead, physically block the noise’ s input source at the file organization layer.**

This mechanism introduces no middleware code, requires no model upgrade, builds no graph network—it only adjusts the physical organization structure of files. This makes it a purely **consumer-side design**: it adds no new constrained input to AI, only manages the information boundary AI activates when each session starts.

The establishment of this principle came from direct practical recognition. The Owner identified in Session 215:

“In the extended cognition system, decomposition is your core capability—grounded in understanding purpose, and actively chosen. The extended cognition system is nothing more than a task to be solved. That task’ s accurate description: for a specific purpose, build a specific information indexing system, and prepare to minimize context space requirements during task execution.”

The project specifications also record a counterexample from before this principle was established: in the early stage when DEVNOTES simultaneously carried both “settled decisions” and “process discussion brainstorming,” when AI reloaded the file in subsequent sessions, it could not distinguish between “conclusions” and “rejected discussion options,” and incorporated deprecated schemes into new design outputs. The project engineering specification document (bang-v3/plan-csf-v2/protocols/closing-discipline.md § 6) explicitly records this as a counterexample: “DEVNOTES simultaneously carrying ‘decision summary’ and ‘process discussion

log' → cognitive-layer baseline contaminated by process information. Correction: decision summary retains only conclusions; discussion moved to execution-layer log.” This direct observation catalyzed the core writing specification at the heart of Baseline-Log Separation.

4.2 Mechanism Design

Baseline-Log Separation serves two parallel purposes, neither dispensable.

First, **ensure working agents contact only current concepts, uncontaminated by history**—the reason the baseline exists. The baseline uses overwrite-mode updates, retaining only current truth at any moment, carrying no trace of historical discussion.

Second, **decision process continuity must not be broken**—the reason session logs exist. Subsequent workers need to know how a decision was made, what options were considered, why some were eliminated. Without process, every historical decision becomes a black box—whenever a new situation requires backtracking, one cannot understand why something was done, nor evaluate whether it can be adjusted.

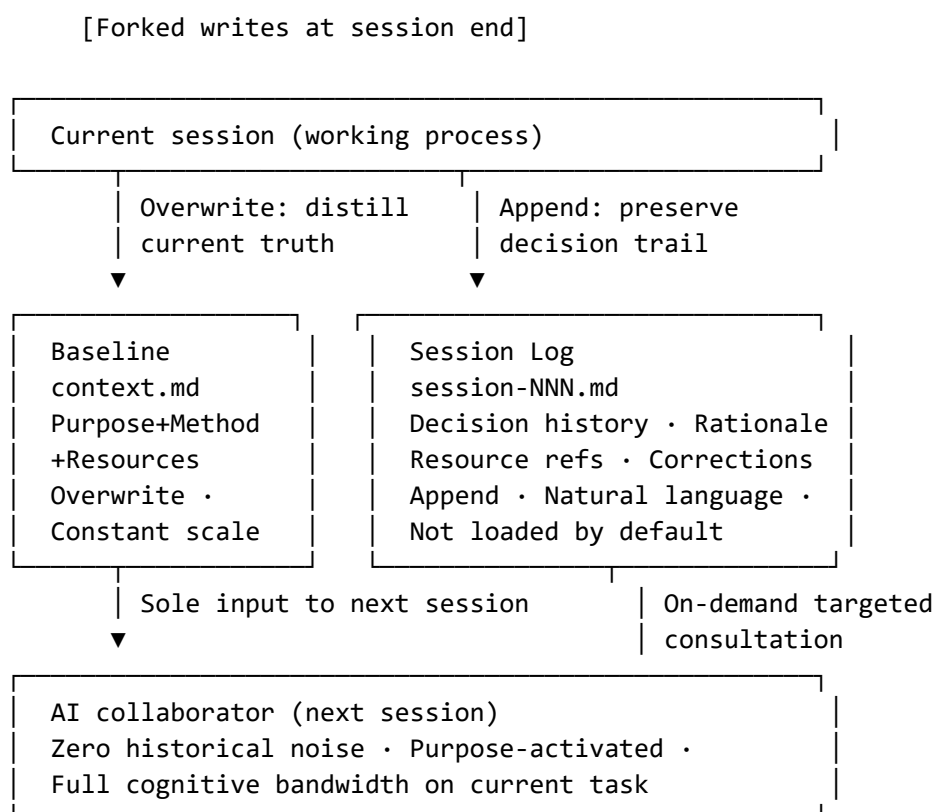
As the Owner put it: **“Move the current state forward, and a log naturally appears.”** Baseline and session log are not two independently maintained systems—they are slices of the same information body across two temporal dimensions: the baseline is the “cross-section of now” ; the session log is “the path walked.”

Baseline (e.g., context.md) — Overwrite-mode updates. The baseline stores only the current task triple: **Purpose + Method + Resources**. Purpose comes first—the AI first understands purpose, then selects methods based on its own knowledge, then organizes just-sufficient resources. After each task, the baseline is overwritten with the new state, retaining no history. This locks new-session cold-start input at constant scale; startup cost does not grow regardless of how many sessions the project has undergone.

Session Log (e.g., session-NNN.md) — Append-mode recording. The session log's purpose is **reconstructing the context of each decision and action**. It records each session's process in append mode: how decisions formed, alternatives considered and eliminated (with rationale), resource references (including filenames, folder names, section names, and other symbolic indices), and business-understanding corrections arising from the work. Natural-language writing is a hard rule—not excluding the appearance of symbolic indices like filenames, but requiring every entry to have natural-language explanatory support, enabling subsequent workers to reconstruct context through reading comprehension rather than mechanical retrieval.

Natural-language writing is the key. When backtracking is needed, what AI reads is not keyword indices but the complete decision context of that moment—why this was done, what options existed, why one path was chosen. AI judges from contextual understanding, not keyword matching; when understanding is correct, conclusions are naturally accurate. This is the essential mechanism by which session logs function—and is the temporal extension of natural language’s advantage as an information carrier over symbolic systems.

Session logs do not physically enter new-task context windows, blocking interference from deprecated content on new tasks.



4.3 Results

By moving historical incident-response patches and symbolic index systems out of the context, post-Session 215 AI Instructions dropped from **308 lines to ~80 lines** (~75% reduction), achieving “thin-shell” status. This independently corroborates SkillReducer [10], which validated a similar principle at the agent tool-description layer: iteratively pruning the tool candidate set and simplifying descriptions significantly improves task execution accuracy—danger comes from redundant information in the window, not insufficient information.

It is worth noting that this project went through **two rounds** of specification simplification. The first round (“SP-Trim,” ~Sessions 136–141) applied a quantitative reduction strategy, cutting specification lines from ~1,100 to

~480 (↓ 56%), yet the core context file (context.md) actually grew from 2,197 to 2,256 lines—and Index Sickness did not disappear. The genuine cliff-drop occurred in the second round: the Baseline-Log Physical Separation described in this paper. This contrast reveals that the effective variable is not the line count of specifications, but the organizational architecture of information. Full discussion in § 5.1.

Key comparisons before and after upgrade:

Dimension	Before (symbol indexing + rule containment)	After (baseline isolation + natural language)
Instructions scale	308 lines (heavy defensive patches)	~80 lines (meta-capability pointers + collaboration discipline)
DEVNOTES decision entries	141+ entries (with B/A/M three-tier classification and reverse index)	5 entries (one-line natural language description, no classification system)
Owner correction density	0.79 corrections/session (#136–#214, 57 session files)	0.53 corrections/session (#253–#395, 38 session files), ↓ 33%
Phantom Legislation	Recurrent (Session 215 representative)	No instances matching Index Sickness failure patterns were observed in the subsequent ~150 session logs
Human correction capability	Symbol-dependent; intuition cannot intervene	Natural language; Owner can verify by intuition at any time

Note: This project did not systematically record per-session token consumption; context windows were generally 160K–200K tokens, and sessions typically ended before window exhaustion, so quantitative context-load comparison cannot be extracted from the logs. However, the session logs and session records preserve an observable qualitative indicator: the operational meaning of “no instances observed” here is that across the subsequent ~150 sessions, the Owner never encountered any session in which “AI claimed completion but physical file verification found it disconnected,” nor did any meta-incident

repair session arise that needed to be dedicated entirely to error correction (of the type of Session 215 in § 2.1). Concretely: after the upgrade, the Owner almost never needed to stop mid-session to point out errors or restate collaboration discipline—the interaction pattern simplified to “start, confirm, end.” Before the upgrade, frequent correction, clarification, and realignment were session norms. This shift is traceable in the session logs and session records and can be treated as a qualitative indicator of substantially reduced collaboration friction. This limitation is a direct consequence of the action research design—data collection served engineering rather than experimental purposes.

The Owner correction density figure in the table above is derived from a systematic statistical analysis of 139 session record files covering sessions #136–#395. We extracted all Owner inputs (user: lines), totaling 1,242 entries, and applied a keyword-matching plus manual review classification method to identify “error correction” and “business course-correction” inputs, with suspected Index Sickness cases individually verified against the operational definition in § 2.1. Classification rules, manual review records, and raw statistical data have been open-sourced alongside the project for independent reproduction (same repository as § 1.2; analysis scripts located under `cases/bang-v3/`).

4.4 Supplementary Case: Cross-Session Provenance Efficiency (Sessions 373–375)

The mechanism’s value received further validation in a systematic gap analysis during the project’s later stage.

Background. At integration testing in Session 373, the team found the “Forwarding Overview Bar” component had never been implemented. On the surface this was a single-point feature gap, but deeper investigation revealed a root cause spanning four design layers: two related design theme packages (TD-02 and Main-5) had advanced independently at different times, and when one was finalized, the other’s business concepts did not yet exist—interfaces never aligned.

Provenance. To locate this temporal disconnect, the AI CoS needed only to read two files based on the relevant task’s resource index: TD-02’s `package.md` and Main-5’s planning file. Both files, per CSF convention, recorded creation dates and session numbers at inception. From these two semantic annotations, the AI gave an immediate precise verdict:

“TD-02 created: 2026-05-19 (Session 138); Main-5 ‘Forwarding Relationship Rendering’ established: 2026-05-21 (Session 168). When TD-02 was finalized, Main-5 simply did not exist.”

This judgment spanned 30 sessions of temporal distance—without scanning 370+ raw session logs, without vector retrieval (RAG), completed within a single session.

Mechanism insight. This case illustrates another dimension of the session log mechanism: physical isolation’s purpose is not only “blocking noise” but equally “preserving context.” Session logs record not a chronological stream but the complete decision context of each action—including purpose, alternatives considered, and rationale for choices—all in natural language, enabling AI when backtracking to reconstruct decision context and judge from understanding rather than keyword matching.

This is the consistent logic of the entire mechanism: not pursuing mechanical precision in AI, but enabling full understanding—when understanding is correct, conclusions are naturally accurate. In Session 373’s case, AI read two files carrying complete decision context to complete temporal localization spanning 30 sessions—not through retrieval technique, but through understanding.

This is consistent with the baseline’s design philosophy: **higher semantic information density is superior to accumulated information volume.**

Looking upward from the tool layer, the core proposition this mechanism reveals is: what determines long-horizon human–AI collaboration quality is not AI memory capacity, but the semantic control structure jointly maintained by the human–AI team. The theoretical implications of this proposition and the larger research landscape are the central topic of § 5.3.

V. Discussion and Conclusion

5.1 Alternative Explanations and Validity Argument

The most prominent alternative explanation for this study is the **team learning effect**: after experiencing intensive failures in the first half of the project, the Owner and AI collaborator may have naturally become more cautious in their collaboration patterns—rather than the Baseline-Log mechanism itself bringing improvement. This paper’s observations cannot

strictly exclude this effect, because “mechanism introduction” and “experience accumulation” coincide temporally. We present four traceable counter-arguments.

Counter-argument 1: The structural fact of two rounds of simplification.

As described in § 4.3, the project had already undergone one round of quantitative reduction (“SP-Trim,” ~Sessions 136–141) before the CSF upgrade. The results of both rounds are compared below:

	Round 1: SP-Trim (~Sessions 136–141)	Round 2: Baseline-Log Physical Separation (~Sessions 215–252)
Core strategy	Quantitative reduction (delete rules, merge files, cut lines)	Change information architecture (baseline overwrite / log append)
Specification line change	~1,100→480 lines (↓ 56%, with quantitative acceptance record)	Instructions 308→80 lines (↓ 75%)
context.md lines	2,197→ 2,256 (+2.7%, actually grew)	680→ 189 (↓ 72%, cliff- drop)
DEVNOTES decision entries	141+ entries (unchanged)	141+ entries→ 5 entries (↓ 96%)
Post-upgrade Index Sickness	Still erupted (#204 severe error, #205 restart, #215 Phantom Legislation, #221 failure)	0 cases (none observed across subsequent ~150 sessions)

At that point, the team had already accumulated ~140 sessions of collaboration experience, and the specification volume had already been cut by more than half. If improvement came from “experience accumulation,” it should have been observable after Round 1—but in fact, Index Sickness erupted in a concentrated burst across the ~60–80 sessions following Round 1, and only disappeared after Round 2 changed the information organization structure. The improvement corresponds to a qualitative change in information architecture, not to accumulated experience or reduced rule count.

Counter-argument 2: Quantitative shift in correction density. As reported in § 4.3, systematic statistics over 1,242 Owner inputs show correction density dropped from 0.79 corrections/session pre-upgrade to 0.53 post-upgrade (↓ 33%); of the 45 genuine corrections pre-upgrade, 9 (20%) met the

Index Sickness operational definition; of the 20 post-upgrade, 0 did. The learning effect may explain a natural decline in total corrections, but it cannot explain “cases meeting the Index Sickness operational definition dropping from 9 to zero” —the latter corresponds to a structural disappearance of a specific failure mode, not a gradual improvement in overall quality.

Counter-argument 3: Qualitative shift in session titles. Pre-upgrade session logs (before Session 215) frequently bear titles such as “Failure,” “Severe Error Discovered,” “Start Over—Redefine Collaboration Norms,” and “Talked Into a Dead End” ; post-upgrade titles are uniformly simple and productive (“Development”), with none recording correction or reconstruction episodes.

Counter-argument 4: Qualitative shift in Owner input content. Pre-upgrade, the Owner frequently issued multi-point corrective instructions (canonical form: “I have three issues: 1. Engineering-level question... 2. You need to fully understand [the theory] before proceeding—don’ t just mechanically execute... 3. Analyze the problem in the business context”), and repeatedly interrupted sessions to address “severe errors” ; post-upgrade, Owner inputs contracted to brief directional prompts (“Let’ s start,” “Continue”), with no diagnosis or correction content required. If this improvement were due to accumulated prompting skill, we would expect prompts to become more *refined* but remain the same *type* (correction still present, just more precise); what was actually observed is that the *necessity* of corrective input disappeared altogether. This qualitative shift differs mechanistically from the learning effect’ s expected pattern of “handling the same type of problem more skillfully,” and is traceable in the session records. The improvement was not gradual—also inconsistent with the gradual nature of learning effects.

Of the four counter-arguments, the first two are derived from quantifiable project records and the latter two from qualitative log observations; all four are independent of each other, and all point to the same conclusion: the improvement corresponds to a structural change in information architecture, not a natural accumulation of collaboration experience. That said, multi-project comparison studies remain the ultimate path to statistically distinguishing the two effects.

5.2 Research Limitations

Methodological Limitations

This study uses single-case action research, with data from one project (Bang-v3) over a specific time period. Action research has recognized research value in software engineering [11][12], and Treude and Storey note that in software

engineering research involving deep LLM participation—where “AI is not only a tool but an active collaborator” —the researcher-as-practitioner action research framework captures cognitive phenomena in real interaction better than traditional controlled experiments [13]. The generalizability of conclusions requires validation across more projects, more collaborators, and more LLM versions. Some effect indicators come from participant observation rather than controlled quantitative measurement; subsequent research can design more rigorous evaluation protocols.

Reflexivity Note. The first author is simultaneously the project Owner, the designer of the CSF mechanism, and the evaluator of the outcomes reported in this paper. This triple role carries the risk of confirmation bias—the Owner had a motive to see the mechanism they designed validated by the project they ran. The bias-reduction measures adopted include: (1) session records from the 136 sessions onward are fully preserved for independent third-party verification, rather than selectively presented; (2) the evaluation baseline is not subjective satisfaction but traceable engineering facts—Instructions line count is measurable, correction events within sessions are locatable, and Phantom Legislation cases include `grep` command records of physical file verification; (3) the Baseline-Log Separation mechanism itself requires the baseline to contain only conclusions and session logs to contain the full process—this physical constraint also limits the Owner’s ability to revise the narrative post hoc. Despite these measures, confirmation bias cannot be entirely eliminated; subsequent multi-project, multi-Owner independent validation is necessary.

Theoretical Scope

The axiom that “natural language outperforms symbols” holds within the context of LLMs pretrained primarily on natural language. For specialized models pretrained primarily on code or structured data, the scope of this axiom requires separate evaluation.

5.3 Conclusion

This paper’s most important finding is not that “symbol systems sometimes fail” —engineering practitioners have long had this intuition. What truly deserves examination is the **universality and unconscious nature** of this failure: in the first half of our project (before the upgrade around Session 222) and in the academic literature, new industrial methods, and tools that have emerged in recent years, the formal-constraint path is being advanced in a highly self-reinforcing manner—every time frustration arises over AI’s unreliable behavior, the instinctive response is “add another rule,” “refine a symbol further” ; most related papers and industrial tools

propose improvements centered on the same core question: how to optimize the information injected into the context window. No one is deliberately making a mistake, but as a whole, the field is broadly departing from an overlooked common truth: **the primary obstacle to maintaining semantic accuracy is historical noise in the active context that should not be there —not insufficient AI memory.**

This paper's contribution is not discovering this common truth—it was always there. The contribution is: **re-raising and naming it (the Pang Principle), documenting the consequences of violating it (Index Sickness, with complete session records), and demonstrating the effects of returning to it (before vs. after physical isolation: Index Sickness cases from 9 to zero, correction density down 33%).** These three elements constitute a complete consumer-side argument chain: the common truth first, the cost of violating it second, the way back third. Against the backdrop of the supply-side improvement path continuing to escalate, this chain points to a directional alternative—not a negation of existing work, but a complement to a neglected dimension. In recent years, the heavy structured-memory architectures represented by CoMem, REAL, and GAM [3][4][5], and formalization attempts such as Git Context Controller [2], collectively confirm that long-horizon context degradation is a genuine engineering challenge; but their prescriptions—more complex middleware, more precise symbolic structures—run counter to this paper's direction. They operate under the assumption of “how to help AI remember more.” This paper's diagnosis is the inverse: the problem is not that AI remembers too little, but that the active context is saturated with historical noise that should not exist—the divergence in their prescriptions is rooted in the formulation of the problem. Whether the two directions are complementary rather than opposed is not yet answerable from available experimental data—this is among the most immediate topics for follow-up work. The industrial practice direction represented by Context Engineering [6] addresses a supply-side question: how to dynamically select what information to inject into and remove from the context window—including compaction, externalized note-taking, and sub-agent architectures. This is adjacent but non-overlapping with the structural problem identified in this paper (the physical cohabitation of baseline and historical discussion); the two exist as complementary dimensions forming the complete engineering landscape of long-horizon human–AI collaboration.

This finding has a clear home in the larger intellectual landscape. Clark and Chalmers' extended mind theory [14] and Hutchins' distributed cognition research [15] jointly indicate that cognitive processes do not terminate at the boundary of the individual mind but extend into the system constituted by artifacts, documents, and the surrounding environment. In the human–AI

collaboration context, baseline and session logs are the physical carriers of this extended cognitive system—not patches on AI memory, but the shared cognitive control structure of the entire human–AI team. Booch identified, before the LLM era, that software development is fundamentally a social activity: establishing shared mental models across teams is more fundamental than any formal specification [16]. When one member of the collaboration team is a large language model, the weight of that judgment only increases. The engineering findings documented in this paper represent a concrete engineering instantiation of this philosophical position in the human–AI collaboration context—with complete session logs and session records available for external verification.

This study is built on single-project action research data; its generality awaits validation across more projects, collaborators, and LLM versions. The effectiveness boundary between the “active selective forgetting” strategy and heavy memory architectures lacks controlled experimental data—this is the most direct next step. The deeper question: as semantic control structure becomes a core asset of human–AI collaboration engineering, how will its design principles evolve alongside continuous model improvement? What the Pang Principle reveals is not merely a set of specific engineering practices, but a class of problem formulations—a class that becomes more, not less, serious as models grow more capable.

Acknowledgments

The authors warmly thank Shuren Song for his companionship and countless weekend exchanges of ideas; his theoretical guidance and emotional support were a core driving force behind this work.

This paper was produced under the CSF (Collaboration Specification Framework) Baseline-Log Physical Separation protocol. Throughout the writing process, the human author (H. Zhang) was responsible for purpose activation—stating direction verbally, rendering judgment, and raising corrections; all manuscript prose was generated by the AI collaborator. The authors declare this AI involvement in full, consistent with the methodological declaration in § 1.3.

References

Formalization Constraint Path

[1] Chen, Z. et al. (2026). “Promptware Engineering: Software Engineering for Prompt-Enabled Systems.” *ACM Transactions on Software Engineering and Methodology (TOSEM)*. arXiv:2503.02400.

[2] Wu, J. et al. (2025). “Git Context Controller: Manage the Context of LLM-based Agents like Git.” arXiv:2508.00031.

Heavy Structured-Memory Path

[3] Zhang, Y., Dong, C., Jin, S., Yu, C., Cui, H., Jin, H., Zhang, X., Bonab, H., Lockard, C., et al. (2026). “CoMem: Context Management with A Decoupled Long-Context Model.” arXiv:2605.30842.

[4] Lu, K., Chen, L., Jiang, G., Qin, Z., Liu, Y., & Zhang, W. (2026). “REAL: A Reasoning-Enhanced Graph Framework for Long-Term Memory Management of LLMs.” arXiv:2606.10694.

[5] Wu, Z., Zhang, H., Lin, F., Xu, W., Xu, X., Chen, Y., Zou, H.P., Chen, S., Zhang, W., et al. (2026). “GAM: Hierarchical Graph-based Agentic Memory for LLM Agents.” arXiv:2604.12285.

Context Engineering (Framework Overview)

[6] Rajasekaran, P., Dixon, E., Ryan, C., & Hadfield, J. (Anthropic Applied AI Team). (2025). “Effective Context Engineering for AI Agents.” *Anthropic Engineering Blog*, Sep 29, 2025.

<https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>

Empirical Studies on Long-Context Limitations

[7] Hong, K., Troynikov, A., & Huber, J. (2025). *Context Rot: How Increasing Input Tokens Impacts LLM Performance*. Chroma Technical Report.

<https://trychroma.com/research/context-rot>

[8] Raju, R., Ji, M., Upasani, S., Li, B., & Thakker, U. (2026). “The Limits of Long-Context Reasoning in Automated Bug Fixing.” *ICLR 2026 ICBINB Workshop*. arXiv:2602.16069.

[9] Liu, N.F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). “Lost in the Middle: How Language Models Use Long Contexts.” *Transactions of the Association for Computational Linguistics*, 12, 157–173. <https://aclanthology.org/2024.tacl-1.9/>

Empirical Support for the “Subtraction Strategy”

[10] Gao, Y., Li, Z., Yuanyuanyuan, Ji, Z., Ma, P., & Wang, S. (2026).
“SkillReducer: Optimizing LLM Agent Skills for Token Efficiency.”
arXiv:2603.29919.

Action Research Methodology

[11] Staron, M. (2024). “Teaching Action Research.” *EMSE Edu Book*.
arXiv:2408.02399.

[12] Wohlin, C., Runeson, P., Höst, M., Ohlsson, M.C., Regnell, B., & Wesslén, A.
(2012). *Experimentation in Software Engineering*. Springer.

[13] Treude, C. & Storey, M. (2025). “Generative AI and Empirical Software
Engineering: A Paradigm Shift.” *Alware 2025*. arXiv:2502.08108.

Extended Cognition / Cognitive Science / Socialized Software Engineering

[14] Clark, A. & Chalmers, D. (1998). “The Extended Mind.” *Analysis*, 58(1),
7–19.

[15] Hutchins, E. (1995). *Cognition in the Wild*. MIT Press.

[16] Booch, G. (1994). *Object-Oriented Analysis and Design with Applications*
(2nd ed.). Benjamin/Cummings.
